redis



- Redis (Remote Dictionary Server) 개요
  - 고성능의 키-값(key-value) 저장소로, 거대한 맵(Map) 데이터 저장소형태를 가지고 데이터를 메모리에 저장하여 빠른 읽기와 쓰기를 지원
  - 주로 캐싱, 인증 관리, DB동시성 제어 등에서 다양한 목적으로 사용
- 레디스 주요 특징
  - key-value로 구성된 단순화된 데이터 구조로 sql 쿼리 사용 불필요
  - 빠른 성능
    - 인메모리 NoSQL 데이터베이스로서 빠른 성능
      - rdb는 기본적으로 disk에 저장이고 필요시에 메모리에 캐싱하는 것이므로, rdb보다 훨씬 빠른 성능
      - redis의 메모리상의 데이터는 주기적으로 스냅샷 disk에 저장
    - key-value는 구조적으로 해시 테이블을 사용함으로서 매우 빠른 속도로 데이터 검색 가능
  - Single Thread 구조로 동시성 이슈 발생X
  - 윈도우 서버에서는 지원하지 않고, linux서버 및 macOS등에서 사용 가능

- 자료구조

  redis 주요 스크립트

  - String, Lists, Sets, Sorted Sets, Hashes 등의 자료 구조를 지원
  - redis에서 모든 key값은 문자열이고, value 또한 문자/숫자 구분 없이 모든 데이터를 문자열 형식으로 저장
  - **Strings**
    - 데이터를 String형태의 value로 저장
    - 가장 일반적인 key - value 구조의 형태
  - **Lists**
    - 순서가 있는 문자열 목록
    - deque(double-ended queue)와 유사한 구조
  - **Sets**
    - 중복을 허용하지 않는 문자열 집합
  - **Sorted Sets**
    - 점수가 할당된 문자열로 이루어진 집합, score를 기준으로 정렬된 순서로 관리
  - **Hashes**
    - Hash는 value값이 map자료구조

- 설치 및 기본명령어
  - 윈도우 및 mac os에서의 설치
    - docker desktop 설치 → 모두 default로 설치
    - 터미널창 open 후 docker run --name redis-container -d -p 6379:6379 redis
  - 리눅스에서의 설치
    - sudo apt-get update
    - sudo apt-get install -y redis-server
      - redis-server --version
    - 서버 시작
      - sudo systemctl start redis-server
  - 접속
    - docker로 설치한 redis접속
      - docker ps → container ID확인
      - docker exec -it <containerID> redis-cli
    - 리눅스
      - redis-cli
  - redis 데이터베이스는 0~15까지로 16개로 구성
    - select 데이터베이스숫자
    - 최초 접속시 default 0번

- 주요 자료구조별 특징과 주요 명령어
  - 모든 key값 조회
    - keys *
  - 키 삭제
    - DEL key
    - 전체 삭제는 FLUSHDB(현재 데이터베이스의 모든 key삭제)
  - String관련
    - 키에 값을 설정
      - SET key value
    - 키의 값을 가져옴
      - GET key
    - NX 문법
      - key값이 존재하지 않는다면(if not exists) 값 setting
      - set key value nx
    - EX 문법
      - “set key값 value값 nx ex 초단위시간” 형식으로 key유지(만료) 시간 세팅
      - “EXPIRE key값 3600” 이런식으로 별도 부여도 가능
      - TTL(Time To Live)이라 부르기도 함.
        - “ttl key값” 명령어를 통해 남은 만료시간 확인 가능
    - 활용
      - 좋아요 기능 구현
      - 재고 처리(동시성 이슈 해결)
      - 캐싱처리(json형식의 데이터를 value값으로 많이 사용)

- list
  - deque 또는 double-ended queue 와 유사한 구조
  - 데이터 추가
    - LPUSH key value
    - RPUSH key value
    - 데이터 중간에 삽입 불가
  - 데이터 추출 : LPOP key, RPOP key
  - 데이터 개수 조회 : LLEN key
  - 활용
    - 웹사이트 최근방문, 최근 살펴본 상품 리스트 등
- SET
  - set은 순서가 없고 중복이 없는 자료구조
  - set(집합)에 멤버 추가
    - SADD myset member
  - set(집합)의 모든 멤버 반환
    - SMEMBERS myset
  - set의 멤버 개수 반환
    - SCARD myset
  - 특정 멤버 삭제
    - SREM myset member1
  - 활용
    - 매일 방문자수 계산
    - 좋아요 수 중복없이 집계

- zset(정렬된 집합)
  - 정렬의 기준이 되는 score를 가지고 있는 set
  - ZADD key score member
    - score : 멤버를 정렬하는 데 사용되는 점수
  - ZREM key member
    - 특정키의 특정멤버 삭제
  - ZRANK key member
    - 특정멤버의 위치 정보 반환
  - ZRANGE stock_prices 0 -1
    - score기준 오름차순 조회
  - ZREVRANGE stock_prices 0 -1
    - score기준 내림차순 조회
  - ZRANGE my_key 0 -1 WITHSCORES
    - withscores가 없을때는 값만 오름/내림차순으로 반환. withscores가 있을 때는 score까지 함께 반환
  - 활용
    - 주식, 코인 등의 실시간 시세저장 또는 게임 등의 사용자의 점수나 순위를 관리
    - 최근 살펴본 상품(리스트 중복제거)

- hashes
  - value값이 map형태인 자료구조
  - 일반 문자열 저장과의 비교
    - json형식의 객체 데이터를 일반문자열로 set하게 될경우, 특정 요소 수정/삭제시 전체 데이터를 변경
    - hash는 map형식의 자료구조를 활용해 특정 데이터만 수정/삭제 용이
  - 주요 메서드
    - HSET
    - HGET
    - HGETALL
    - HINCRBY
  - 활용
    - 빈번히 변경될 가능성이 있는 객체 형식의 데이터 캐싱
      - hash는 특정 요소값만을 변경하기 용이
      - json형식의 문자열의 경우 데이터를 일일이 parsing 후 재 setting해야 하는 비효율
- redis pub sub 기능
  - redis를 활용하여 메시지를 발행하고 구독하는 서비스
  - 특징